

M11 — What this block gets you

- ✓ BQML seen, not drilled— a model trained by CREATE MODEL, living next to your tables
- ✓ The end-to-end closed: extracts → BigQuery → Bronze/Silver/Gold → the Power BI you already have
- ✓ The Import vs DirectQuery call, per mart— default: Import + refresh aligned to Dataform releases
- ✓ Auth the right way— author OAuth on the desktop, a least-privilege service account in the service
- ✓ The gateway answer for M&O— BigQuery is a cloud source; nothing new to run

This block, and what follows

14:35–14:50

Break

14:50–15:05

BQML live: three SQL statements, one trained model — show it exists, don't drill

DEMO

15:05–15:20

The Power BI last mile: connect, Import vs DirectQuery, auth (custom segment)

15:20–15:50

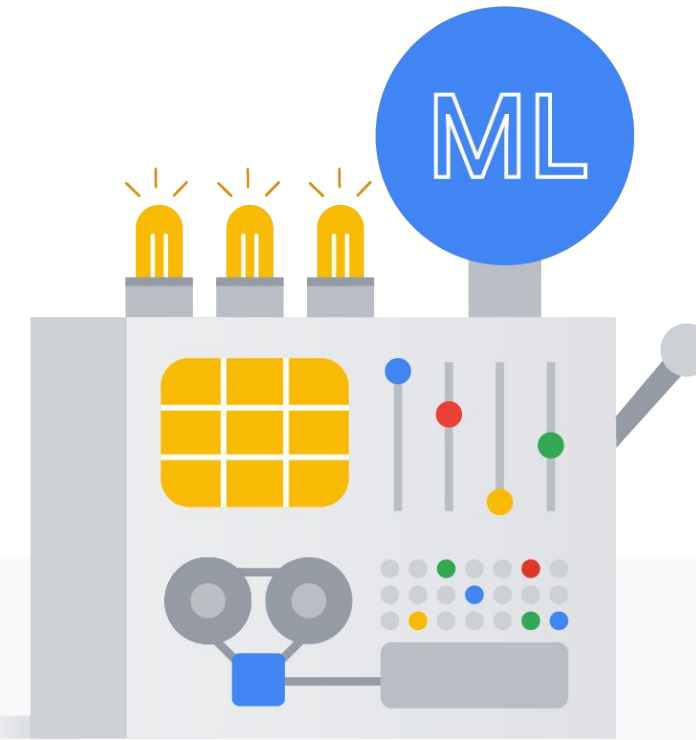
Capstone workshop: map one of your real extract jobs onto the GCP target

WORKSHOP

BigQuery ML



Build and deploy ML models using simple SQL queries.



Accessible ML

You do not need to be an expert in Python or TensorFlow to create valuable predictive models.

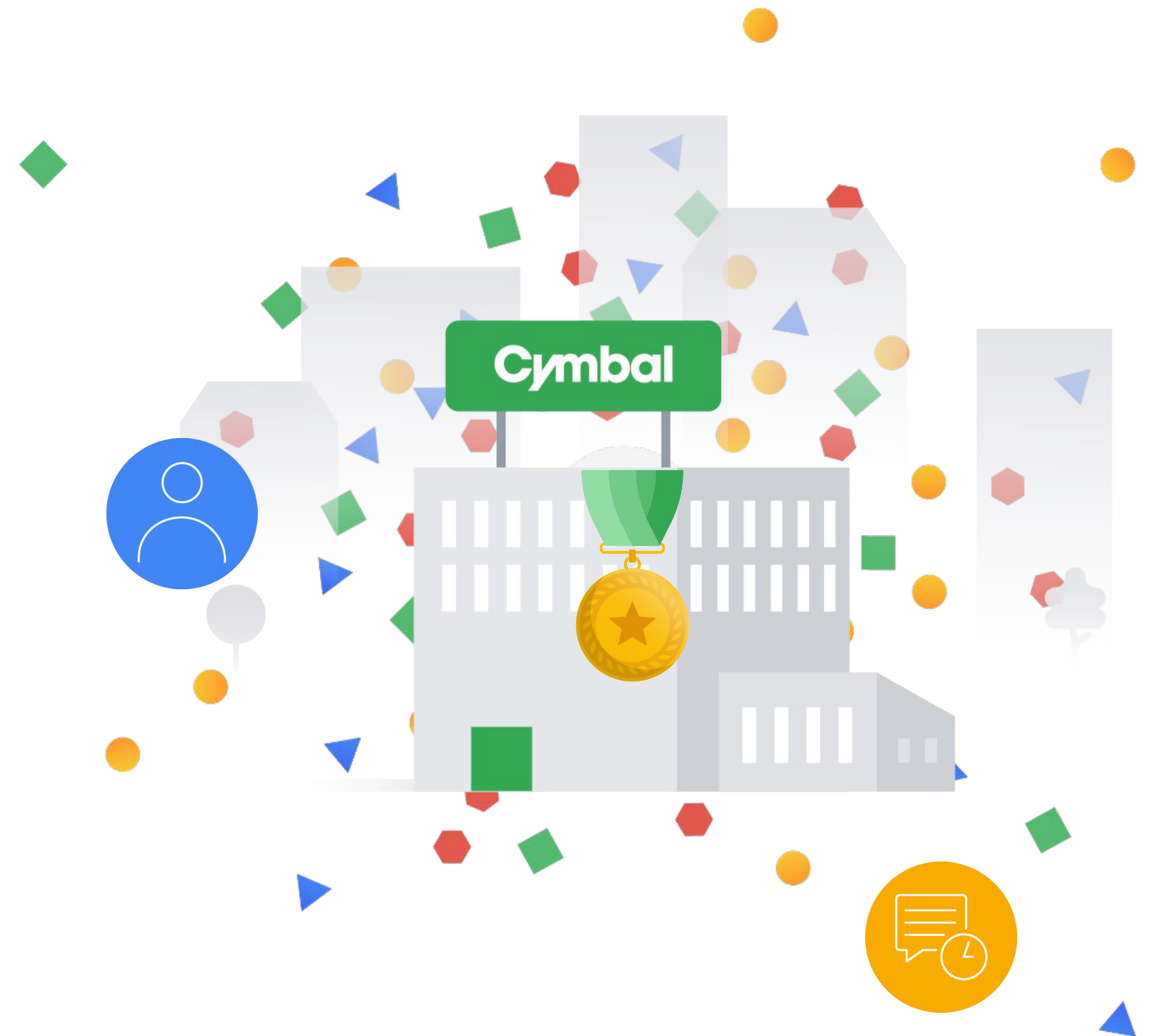
ML where the data lives

Same rule as everything since 9 AM: don't move the data to the tool.

- No export, no notebook— the model trains on data that never leaves BigQuery
- Models are first-class objects— they sit in your dataset, take IAM, appear in Information Schema
- ML.PREDICT is a function call— predictions are a SQL query, schedulable like any other
- Honest scope— ML is a possible future for DOM DoIT, not the priority; the warehouse is
- Why show it at all— when a data-science team arrives, your Gold marts are their training data

Use case

To identify customers who are at risk of not making another purchase, the marketing analytics team can use BigQuery ML and a few SQL statements.



Use case: Predicting customer churn (1 of 4)

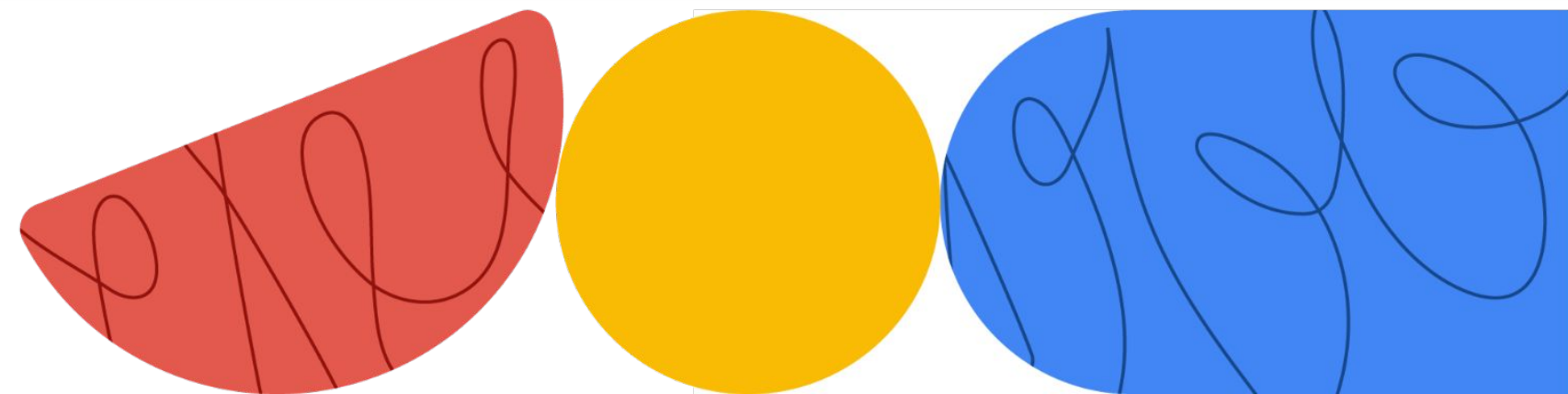
01

Feature engineering

The first step is to prepare the data. The analysts write a SQL query to create features (or signals) that might predict churn.

Fields

- `recency`: days since last purchase
- `frequency`: purchases in the last year
- `monetary_value`: total spent
- `days_since_first_purchase`: customer tenure



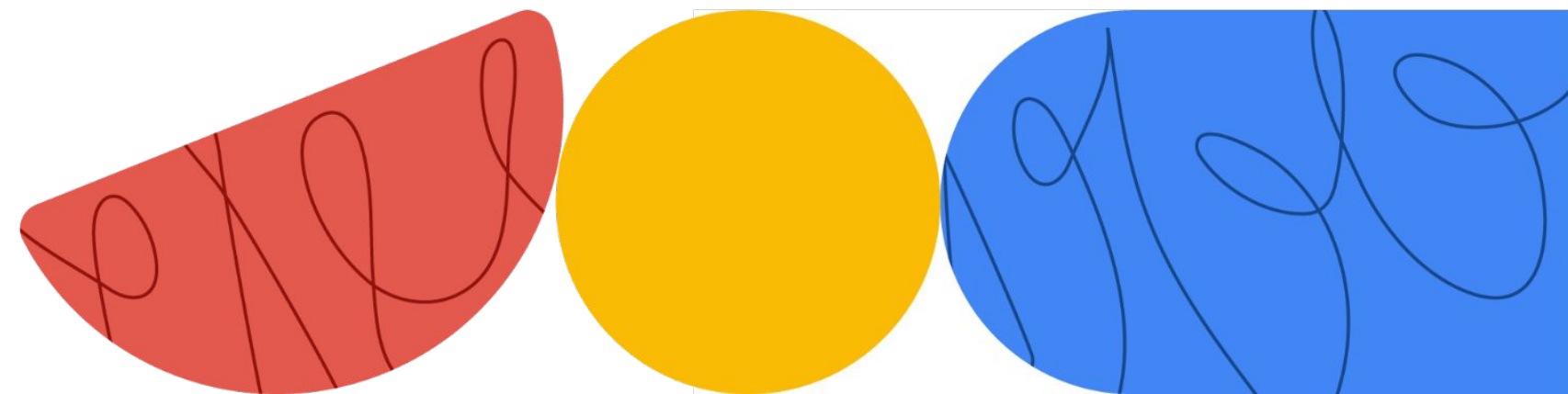
Use case: Predicting customer churn (2 of 4)

02

Model training

After the features are ready, they train a model with a single `CREATE MODEL` statement in SQL.

For this binary classification problem (churn or no churn), a logistic regression or a boosted tree model is a good choice.



Sample code

SQL

```
CREATE OR REPLACE MODEL cymbal_ecommerce.customer_churn_predictor
OPTIONS(model_type='LOGISTIC_REG') AS
SELECT
customer_id,
recency,
frequency,
monetary_value,
(total_purchases > 1) AS will_return -- This is our label
FROM
cymbal_ecommerce.customer_purchase_summary;
```

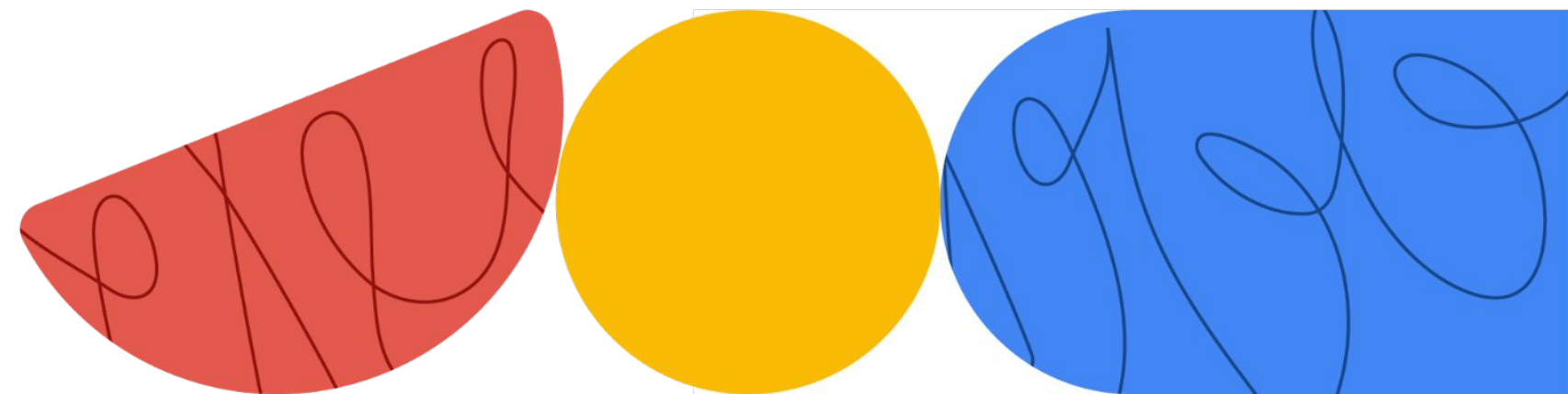
Use case: Predicting customer churn (3 of 4)

03

Model evaluation

After the model is trained, the analysts evaluate its performance using the `ML.EVALUATE` function.

- ✓ Accuracy
- ✓ Precision
- ✓ Recall



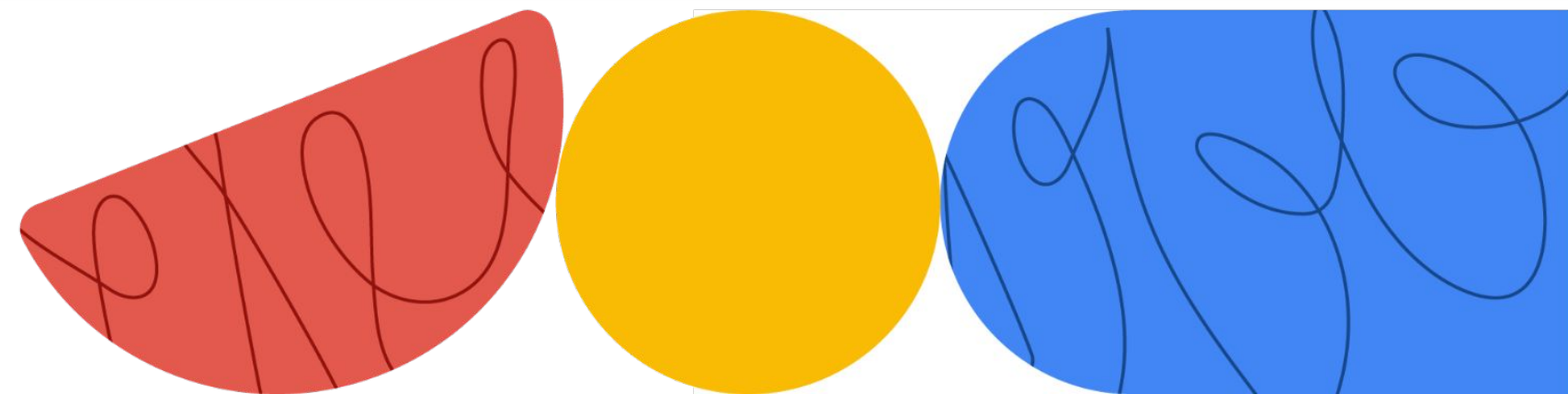
Use case: Predicting customer churn (4 of 4)

04

Prediction

With the `ML.PREDICT` function, you can get a list of all customers and their probability of churning.

This list can then be used to create targeted marketing campaigns to re-engage at-risk customers.



BigQuery ML allows the teams who are closest to the data to **build predictive models**, which leads to faster and more relevant insights.

Demo: a live BQML model in three SQL statements

15 minutes · BigQuery console — instructor's demo dataset

1. Peek at `bigquery-public-data.ml_datasets.penguins` — 344 penguins; we predict `body_mass_g`
2. `CREATE OR REPLACE MODEL class.penguins_model` — linear regression, trains in ~1–2 minutes
3. `ML.EVALUATE` — mean absolute error in plain speech: a unit test for a model
4. `ML.PREDICT` — actual vs predicted side by side, sorted by worst error
5. Point at the Explorer pane — the model sits in the dataset like a table, with IAM

Why it matters — nothing in your brief points at ML — this exists so you know the door is there: the day someone asks about forecasting, the data and the tool already live in the same place.

The last mile: Power BI ↔ BigQuery

Your reports don't change — their source does. The course never mentions this; your project lives on it.

- Get Data → Google BigQuery— the connector ships in Power BI Desktop; the same one Excel uses
- Sign in with your own Google account (OAuth)— BigQuery sees the person, not a shared login
- Navigator shows project → dataset → table— the console hierarchy, inside your existing BI tool
- IAM on the Gold dataset is the gatekeeper— row/column security applies per report author
- From Load onward it's the Power BI you know— DAX, visuals, publish; nothing report-side changes
- Bookmark Microsoft's connector docs page— the pattern is stable, the dialogs evolve

Import vs DirectQuery — decide per mart

The one choice in the connector dialog that matters.

- Import— data cached in Power BI; fastest reports, full DAX, one query per refresh
- Import fits— nightly-built Gold marts; Dataform rebuilds on a schedule, refresh right after
- Import cost— small and predictable; a post-build refresh matches pipeline reality
- DirectQuery— a live query per visual, per user, per click; current on every interaction
- DirectQuery cost— bills BigQuery scan bytes continuously; on-demand pricing needs watching
- DirectQuery fits— intra-day executive freshness, or a dataset too large to cache

Default: Import + scheduled refresh aligned to the Dataform release schedule. DirectQuery buys freshness the pipeline doesn't have yet — revisit per mart when streaming arrives.

Auth and gateways — two identities, one quiet gateway

Desktop and the Power BI service are different stories. Keep them distinct.

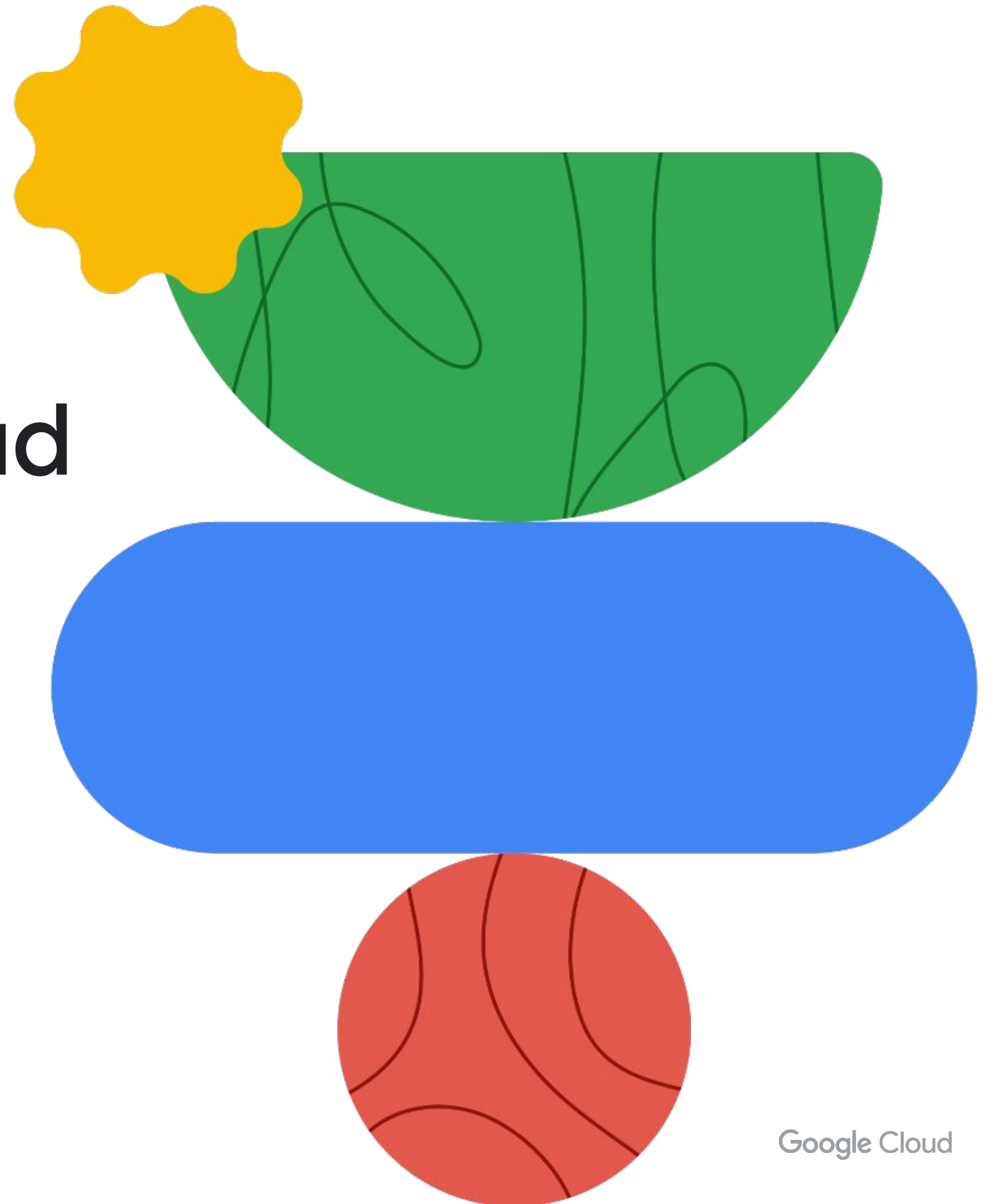
- Desktop— each author signs in with their own Google account; the audit trail is real people
- Service— stored credentials are a service account whose only job is reading the Gold dataset
- Minimum roles— bigquery.dataViewer on the Gold dataset, bigquery.jobUser on the project
- Same discipline as SQL Agent proxies— least-privilege, non-human identity, one job
- Gateway answer for M&O— BigQuery is a cloud source; no on-premises data gateway required
- Gateway shrinks, not grows— only if a report mashes BigQuery with on-prem SQL Server

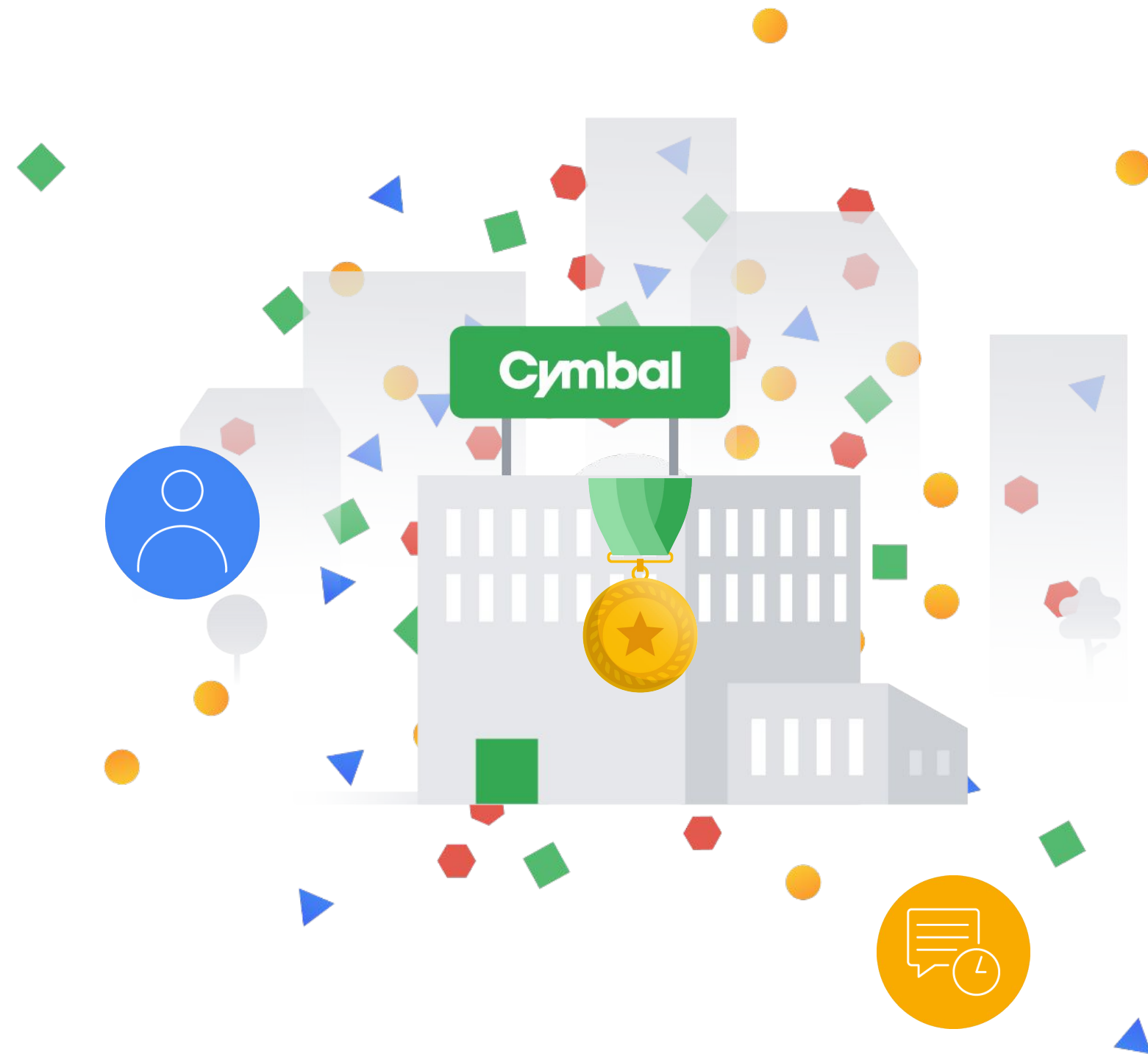
Your Gold marts land in the Power BI you already have. Same authors, same workspaces, same DAX — only the connection string changes.

The close of the Power BI segment — the whole migration, end to end

Building a modern data lakehouse on Google Cloud

Leveraging powerful,
open-source-based services.





Cymbal deals with a massive amount of data daily in various formats. A data lakehouse architecture helps bring it together for powerful analytics and AI-powered customer experiences.

Core components of a Google Cloud data lakehouse

A robust and scalable data lakehouse on Google Cloud is built on a foundation of several key services that work together seamlessly.



Cloud Storage

This is the heart of your data lakehouse, providing a highly scalable and cost-effective object storage service. Think of it as the foundational layer where all your data, in its raw and processed forms, resides.

Cymbal's data

- ✓ Website clickstream data
- ✓ Product images
- ✓ Supplier inventory files (CSV/Parquet)
- ✓ Historical data (Apache Iceberg)

BigQuery

This powerful, serverless data warehouse acts as the analytics engine for your data lakehouse. What makes BigQuery unique is its ability to directly query data in Cloud Storage without needing to move it.

The separation of storage and compute is a core principle. BigQuery provides:

- ✓ Flexibility
- ✓ Cost savings

Knowledge Catalog

As your data grows, managing it can become a significant challenge.

Knowledge Catalog provides an intelligent data fabric that allows you to discover, manage, monitor, govern, and describe your data across your entire data lakehouse.

Cymbal's data

- Knowledge Catalog automatically catalogs data in Cloud Storage buckets, making it easily searchable for data analysts and scientists.
- It enforces data quality rules and security policies, protecting sensitive customer information.

AlloyDB for PostgreSQL

Although the data lakehouse is excellent for analytical workloads, many applications still rely on transactional databases.

AlloyDB is a fully managed, PostgreSQL-compatible database service that offers superior performance and availability.

Cymbal's data

- AlloyDB for e-commerce backend handles real-time order processing and customer account management.
- Integrate AlloyDB data into data lakehouse for broader analytics via federated queries.

Apache Iceberg

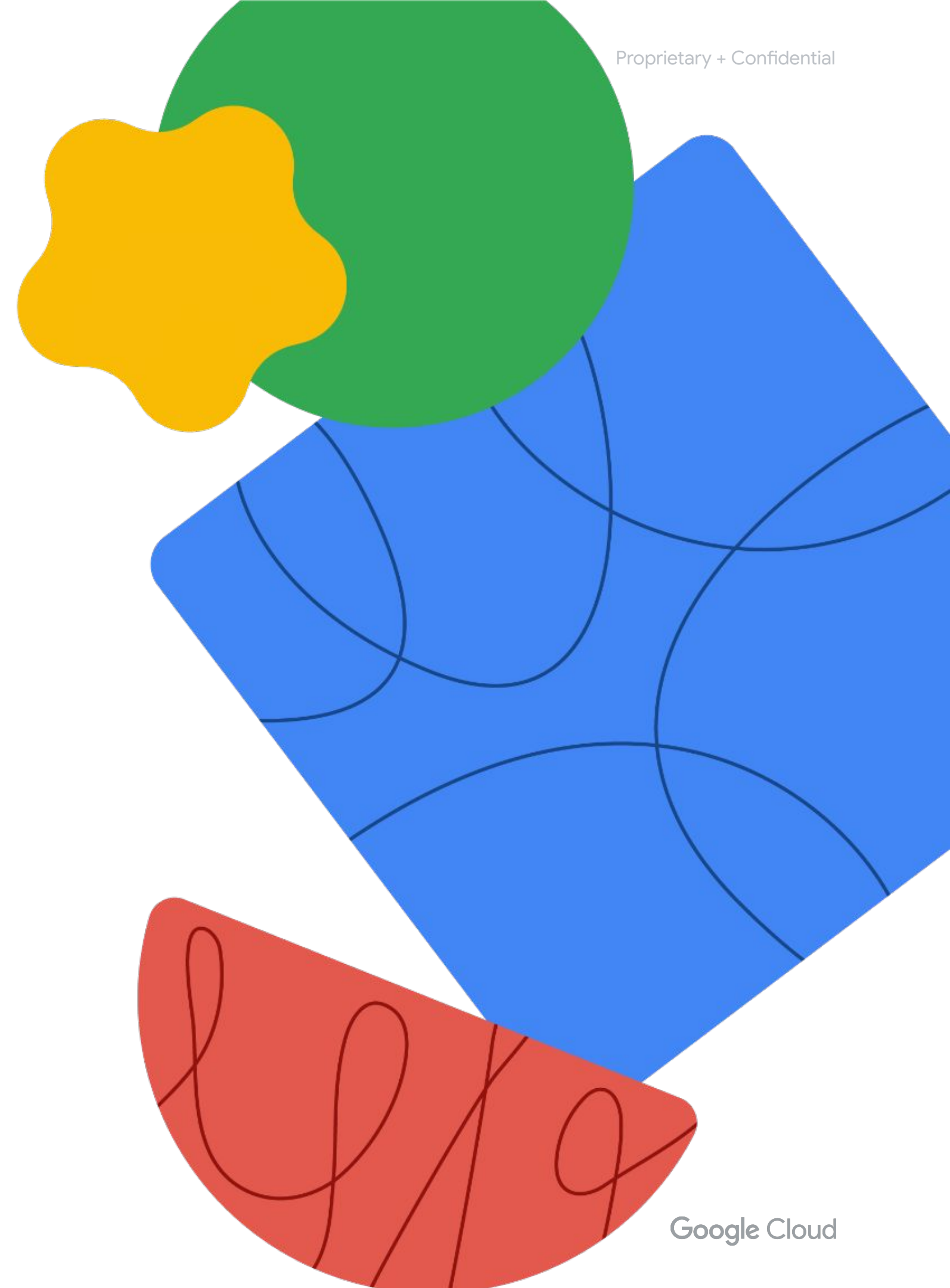
An open-source table format that brings the reliability and performance of a traditional database to your data lake.

Using Apache Iceberg with BigQuery and Cloud Storage, you can perform efficient updates and deletes on your data, a capability that was traditionally difficult in data lakes.

This is crucial for Cymbal when they need to update product information or manage customer data privacy requests.

Architectural patterns and best practices

The most common and effective architectural pattern for a data lakehouse is the medallion architecture, which organizes data into three distinct zones.



The medallion architecture

Bronze zone

This is where data lands in its original, untouched format.

Silver zone

The raw data is cleaned, standardized, and enriched.

Gold zone

Contains highly refined and aggregated data that is ready for business intelligence and analytics.

Best practices

01

Embrace open standards

02

Govern your data from the start

03

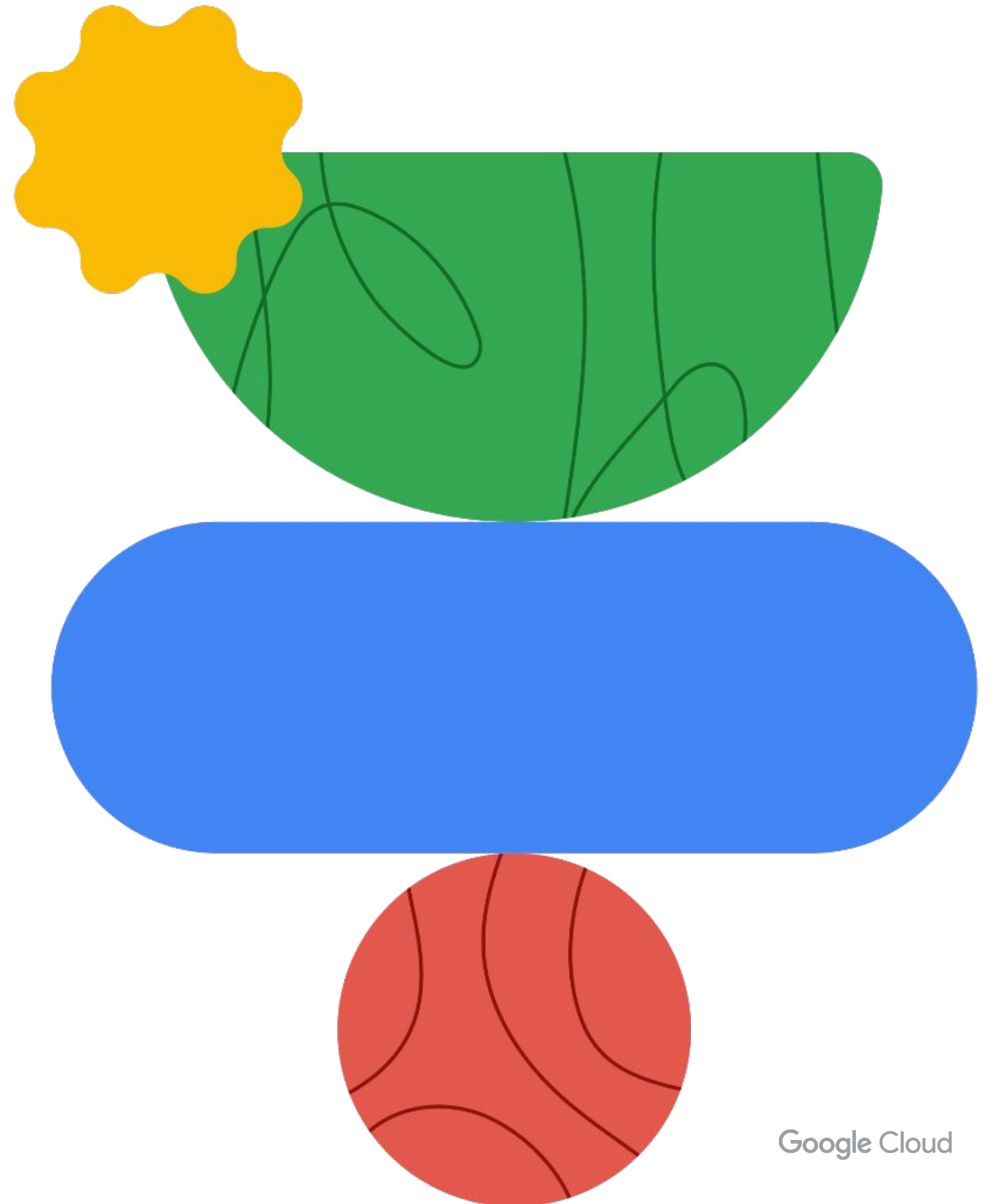
Optimize for cost and performance

04

Automate your data pipelines

Common pitfalls to avoid

Implementing a data lakehouse is a journey, and there are some common challenges to be aware of.



Common pitfalls to avoid

01 Neglecting governance

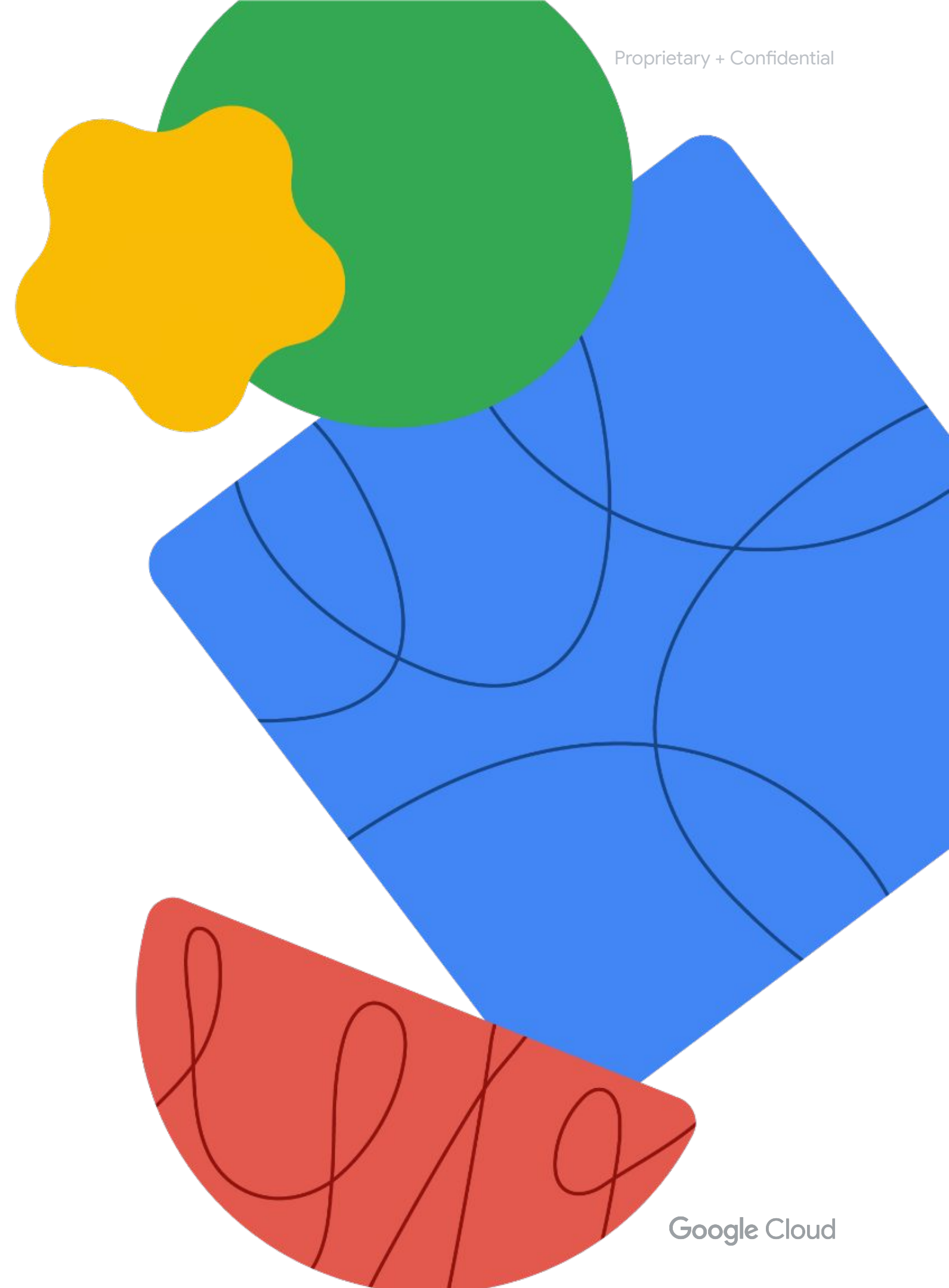
02 Ignoring data quality

03 Vendor lock-in

04 Lack of clear strategy

The future: AI and data modernization

The true power of a modern data platform lies in its ability to support advanced analytics and AI.



AI and data modernization (1 of 2)

01

Advanced analytics can be done quickly using BigQuery ML.

02

Services, such as Agent Platform, can be used to build and train ML models directly on the data in your data lakehouse.

03

Cymbal could develop a recommendation engine for product suggestions or a fraud detection model for suspicious transactions.

AI and data modernization (2 of 2)

The rise of generative AI also opens up exciting new possibilities. Cymbal could use large language models to create engaging product descriptions automatically or to power a chatbot that provides personalized customer support.



M11 — What you now have

- ✓ BQML seen, not drilled— CREATE MODEL → ML.PREDICT in SQL, where your data already lives
- ✓ The end-to-end closed— kill the extracts → BigQuery → Bronze/Silver/Gold → Gold marts → Power BI
- ✓ A default for the marts— Import + scheduled refresh aligned to your Dataform release schedule
- ✓ An auth pattern— author OAuth on the desktop, least-privilege service account in the service
- ✓ Next: the capstone— map one of your real extract jobs onto everything you've just seen